

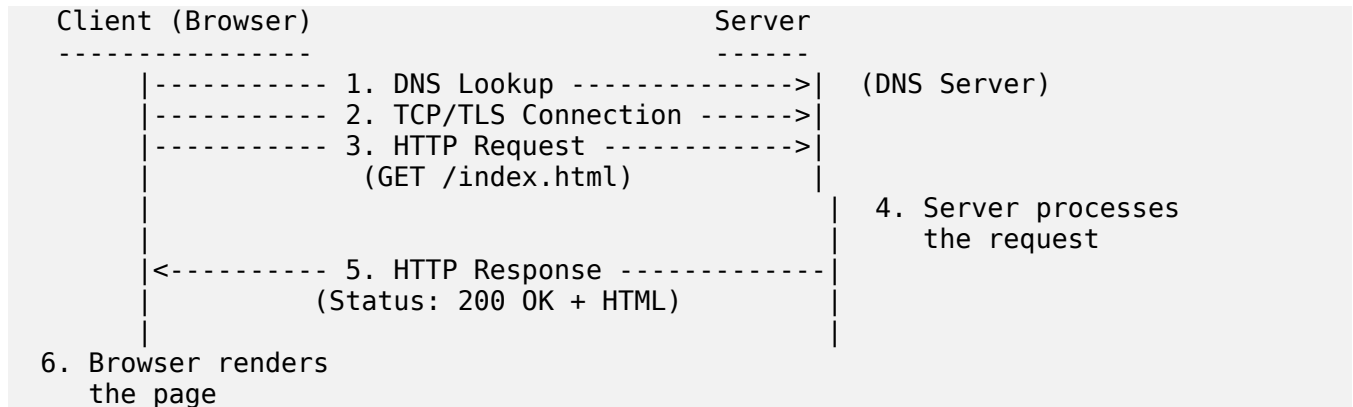
Frontend – Assignment 1

Q1. HTTP Request–Response Cycle

The HTTP Request–Response cycle is how a browser (client) communicates with a web server to fetch a resource. It follows a request-reply model over HTTP/HTTPS. Major steps:

1. DNS Resolution: Browser converts the URL into an IP address using a DNS server.
2. TCP Connection: Browser connects to the server (port 80/443), with an SSL/TLS handshake if HTTPS.
3. HTTP Request: Browser sends a request with a method (GET/POST), URL, headers and sometimes a body.
4. Server Processing: Server processes the request (runs scripts, queries database, locates file).
5. HTTP Response: Server replies with a status code (200 OK, 404, 500), headers and the response body.
6. Rendering: Browser parses the HTML, fetches linked CSS/JS/images, and renders the page.

Diagram:



This cycle repeats for every extra resource (CSS, JS, images) on the page.

Q2. Introduction to Web Technologies

1. HTML

Markup language used to create the structure and content of a webpage using tags like `<p>`, `<div>`, `<img>`.

2. CSS

Used to style HTML elements — colors, fonts, spacing and layout — separating design from content.

3. JavaScript

Client-side scripting language that adds interactivity — form validation, animations, DOM updates without reloading the page.

4. Bootstrap

A CSS framework with ready-made responsive components (grid, navbars, buttons) for quick, mobile-friendly UI building.

5. jQuery

A lightweight JS library that simplifies DOM manipulation, events and AJAX with short, easy syntax ('write less, do more').

6. React.js

A JS library (by Meta) for building component-based UIs using a Virtual DOM for fast, efficient updates.

Q3. Client-Side vs. Server-Side Technologies

Client-side technologies run in the user's browser and control what the user sees/interacts with — e.g. HTML, CSS, JavaScript, React.js, jQuery.

Server-side technologies run on the web server, handling logic, databases and generating the response — e.g. PHP, Node.js, Python (Django), Java (Spring).

Key differences:

- Execution: Client-side in browser; server-side on the server.
- Visibility: Client-side code is viewable via 'View Source'; server-side code stays hidden.
- Example: JS validates a login form on the client; the server checks credentials against the database.

Q4. Block-Level Elements in HTML

Block-level elements always start on a new line and take up the full available width by default.

Characteristics:

- Start on a new line.
- Occupy full width of the parent by default.
- Can hold other block or inline elements.
- Margins, padding, width, height can be set freely.

Examples: <div>, <p>, <table>, <h1>, .

Q5. Semantic HTML Elements

Semantic elements are tags that clearly describe their meaning/purpose (unlike generic `<div>/<span>`).

Importance:

- Accessibility: helps screen readers navigate the page.
- SEO: search engines rank meaningful content better.
- Readability: code is easier to understand and maintain.

Examples: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<footer>`.

Q6. Creating a Table Using HTML

HTML code:

```
<table border="1">
  <thead>
    <tr>
      <th>Sr. No.</th>
      <th>Name</th>
      <th>Designation</th>
      <th>Department</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <td>John</td>
      <td>Manager</td>
      <td>Sales</td>
    </tr>
    <tr>
      <td>2</td>
      <td>Smith</td>
      <td>Sr. Manager</td>
      <td>Marketing</td>
    </tr>
    <tr>
      <td>3</td>
      <td>Jane</td>
      <td>Manager</td>
      <td>HR</td>
    </tr>
  </tbody>
</table>
```

Output:

Sr. No.	Name	Designation	Department
1	John	Manager	Sales
2	Smith	Sr. Manager	Marketing
3	Jane	Manager	HR

Q7. Creating a Nested List Using HTML

'River' is the main item, with Ganga, Yamuna, Brahmaputra and Narmada nested under it as a sub-list:

```
<ul>
```

```
<li>River
  <ul>
    <li>Ganga</li>
    <li>Yamuna</li>
    <li>Brahmaputra</li>
    <li>Narmada</li>
  </ul>
</li>
</ul>
```

Output:

- River
 - Ganga
 - Yamuna
 - Brahmaputra
 - Narmada

Q8. Types of CSS

1. Inline CSS

Applied directly on an element using the style attribute. Quick but not reusable.

```
<p style="color:blue;">Hello World</p>
```

2. Internal CSS

Written inside a <style> tag in the <head>. Applies to that page only.

```
<style>
  p { color: blue; }
</style>
```

3. External CSS

Written in a separate .css file and linked via <link>. Best for reusing styles across pages.

```
<link rel="stylesheet" href="style.css">
```

Q9. Practical Understanding

- HTML (Structure): Defines the content — headings, text, images, forms.
- CSS (Presentation): Controls how content looks — colors, fonts, layout.
- JavaScript (Behaviour): Adds interactivity — clicks, validation, dynamic updates.

Analogy: HTML is the house structure, CSS is the paint/decor, JavaScript is the electricity that makes it functional.

Q10. Short Answer

a. Bootstrap:

Provides ready-made responsive components and a grid system, helping build mobile-friendly sites quickly without writing CSS from scratch.

b. jQuery:

A JS library that simplifies DOM manipulation, events and AJAX. Popular because it reduced code and fixed cross-browser issues.

c. React.js:

A component-based UI library using a Virtual DOM to update only changed parts of a page, making apps faster and easier to maintain.